

MEDIOtec

UniRV
Universidade de Rio Verde

TÉCNICO EM INFORMÁTICA

BANCO DE DADOS II

PROF. JÚLIO CÉSAR



PROF. JÚLIO
ANALISTA DE
SISTEMAS

PLANO DA AULA PRÁTICA

1. **INTEGRIDADE DE DADOS**
   CHECK, PK, FK e TRIGGER

2. **SEGURANÇA EM BANCO DE DADOS**
   CREATE USER, GRANT, REVOKE, ROLE

3. **PRIVACIDADE E LGPD**
   AES, hash, auditoria e direitos

COMO VAMOS TRABALHAR

Aula 100% prática, do começo ao fim:

- Digite, execute e observe cada resultado
- Vamos provocar erros de propósito
- Scripts prontos no arquivo Word da aula
- Em duplas: um digita, o outro confere

Ambiente: MySQL 8.0.16+ (Workbench ou terminal)

PREPARANDO O AMBIENTE

```
DROP DATABASE IF EXISTS escola;
CREATE DATABASE escola
  CHARACTER SET utf8mb4;
USE escola;
SELECT VERSION();
```

Use MySQL 8.0.16+: só assim o CHECK é aplicado.

BLOCO 1 — INTEGRIDADE

A **DOMÍNIO**
CHECK, NOT NULL e DEFAULT

B **ENTIDADE**
PRIMARY KEY e UNIQUE

C **REFERENCIAL**
FOREIGN KEY e CASCADE

D **DEFINIDA PELO USUÁRIO**
CHECK e TRIGGER

~85 minutos, com um desafio no final.

SEM INTEGRIDADE — O PROBLEMA

```
CREATE TABLE sem_regras (
  id INT, idade INT, email VARCHAR(100)
);
INSERT INTO sem_regras VALUES
  (1, -5, NULL), (1, 200, 'x');
SELECT * FROM sem_regras;
```

O banco aceitou tudo: id repetido, idade absurda e e-mail nulo.

DOMÍNIO — CRIANDO A TABELA

```
CREATE TABLE aluno (
  idade INT CHECK (idade BETWEEN 14 AND 99),
  email VARCHAR(100) NOT NULL,
  status VARCHAR(10) DEFAULT 'ATIVO',
  matricula CHAR(10)
    CHECK (matricula LIKE 'BD2%')
);
```

Quatro regras de domínio: CHECK, NOT NULL, DEFAULT e LIKE.

DOMÍNIO — TESTANDO (1/2)

```
-- OK: tudo válido
INSERT INTO aluno (idade, email, matricula)
  VALUES (17, 'ana@escola.com', 'BD2-001');
-- ERRO: idade fora do intervalo
INSERT INTO aluno (idade, email, matricula)
  VALUES (200, 'x@x.com', 'BD2-002');
```

O CHECK barrou a idade 200 (erro 3819).

DOMÍNIO — TESTANDO (2/2)

```
-- ERRO: e-mail obrigatório
INSERT INTO aluno (idade, email, matricula)
  VALUES (17, NULL, 'BD2-003');
-- ERRO: matrícula fora do padrão
INSERT INTO aluno (idade, email, matricula)
  VALUES (17, 'y@y.com', 'XYZ-004');
SELECT * FROM aluno;
```

Só a linha válida foi gravada, com status ATIVO.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

DOMÍNIO — REGRA COM ALTER

```
ALTER TABLE aluno ADD CONSTRAINT chk_email
  CHECK (email LIKE '%@%');
-- ERRO: e-mail sem @
INSERT INTO aluno (idade, email, matricula)
  VALUES (17, 'semarroba', 'BD2-005');
-- Próximo passo: recomeçar a tabela
DROP TABLE aluno;
```

ALTER TABLE adiciona regras a uma tabela já criada.

MÃO NA MASSA

UNIDADE 1 — RESTRIÇÃO DE DOMÍNIO

1

Exercício orientado

Crie a tabela produto: nome obrigatório, preço maior que 0 e estoque com valor padrão 0. Insira um produto válido e provoque 2 erros.

Resolução individual, com correção coletiva ao final.

CORREÇÃO — TABELA PRODUTO

```
CREATE TABLE produto (
  id_produto INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(80) NOT NULL,
  preco DECIMAL(10,2) CHECK (preco > 0),
  estoque INT DEFAULT 0
);
-- ERRO esperado: preço zero
INSERT INTO produto (nome, preco)
  VALUES ('Lápis', 0);
```

Também vale testar nome nulo e um produto válido.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

ENTIDADE — CRIANDO A TABELA

```
CREATE TABLE aluno (
  id_aluno INT PRIMARY KEY AUTO_INCREMENT,
  cpf CHAR(11) UNIQUE NOT NULL,
  nome VARCHAR(80) NOT NULL,
  idade INT CHECK (idade BETWEEN 14 AND 99),
  email VARCHAR(100) NOT NULL,
  status VARCHAR(10) DEFAULT 'ATIVO'
);
```

PRIMARY KEY identifica a linha; UNIQUE protege o CPF.

ENTIDADE — TESTANDO O UNIQUE

```
INSERT INTO aluno
  (cpf, nome, idade, email) VALUES
  ('11111111111', 'Ana', 16, 'a@e.com'),
  ('22222222222', 'Bia', 17, 'b@e.com');
-- ERRO: CPF repetido (UNIQUE)
INSERT INTO aluno
  (cpf, nome, idade, email) VALUES
  ('11111111111', 'Caio', 18, 'c@e.com');
```

Erro 1062: Duplicate entry para o CPF já cadastrado.

ENTIDADE — TESTANDO A CHAVE

```
-- ERRO: id_aluno 1 já existe
INSERT INTO aluno
  (id_aluno, cpf, nome, idade, email)
VALUES
  (1, '33333333333', 'Caio', 18, 'c@e.com');
-- AUTO_INCREMENT: veja os ids gerados
SELECT id_aluno, nome FROM aluno;
```

A chave primária nunca se repete e nunca é nula.

CHAVE COMPOSTA — CRIANDO

```
CREATE TABLE turma (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(50) NOT NULL
);
CREATE TABLE matricula (
  id_aluno INT, id_turma INT,
  PRIMARY KEY (id_aluno, id_turma)
);
```

O par aluno + turma não pode se repetir.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

CHAVE COMPOSTA — TESTANDO

```
INSERT INTO turma (nome)
  VALUES ('Info 2A'), ('Info 2B');
INSERT INTO matricula VALUES (1,1), (2,1);
-- ERRO: o par (1,1) já existe
INSERT INTO matricula VALUES (1,1);
-- OK: mesmo aluno, outra turma
INSERT INTO matricula VALUES (1,2);
```

Matrículas atuais: (1,1), (2,1) e (1,2).

PROF. JÚLIO
ANALISTA DE
SISTEMAS

REGRA DE NEGÓCIO — CHECK

```
CREATE TABLE funcionario (
  id_func INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(80) NOT NULL,
  salario DECIMAL(10,2)
    CHECK (salario >= 1518.00)
);
```

Valor do material da aula: ajuste ao salário mínimo vigente.

REGRA DE NEGÓCIO — TESTANDO

```
INSERT INTO funcionario (nome, salario)
  VALUES ('Carla', 2500.00);
-- ERRO: abaixo do salário mínimo
INSERT INTO funcionario (nome, salario)
  VALUES ('Davi', 1000.00);
```

Regras de negócio simples cabem em um CHECK.

REGRA DE NEGÓCIO — EMPRÉSTIMO

```
CREATE TABLE emprestimo (
  id INT PRIMARY KEY AUTO_INCREMENT,
  livro VARCHAR(80) NOT NULL,
  data_retirada DATE NOT NULL,
  data_devolucao DATE
);
```

Vamos proteger a regra de datas com um TRIGGER.

REGRA DE NEGÓCIO — TRIGGER

Uma **trigger** (ou gatilho) em SQL é um bloco de código procedural associado a uma tabela ou visão que é executado **automaticamente**
pelo banco de dados sempre que um evento específico ocorre.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

REGRA DE NEGÓCIO — TRIGGER

Uma trigger é estruturada em 4 pontos essenciais:

Tabela alvo: Onde a ação acontece.
Evento: A operação executada (INSERT, UPDATE ou DELETE).
Momento: Quando roda — BEFORE (antes, para validar), AFTER (depois, para registrar/auditar) ou INSTEAD OF (no lugar da ação original).
Frequência: Quantas vezes roda — por cada linha modificada (FOR EACH ROW) ou apenas uma vez pelo comando inteiro (FOR EACH STATEMENT).

TRIGGER — PARTE 1/2

```
DELIMITER $$
CREATE TRIGGER valida_devolucao
BEFORE INSERT ON emprestimo
FOR EACH ROW
BEGIN
```

O DELIMITER $$ permite usar ; dentro do trigger.

TRIGGER — PARTE 2/2

```
  IF NEW.data_devolucao <
     NEW.data_retirada THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Data inválida';
  END IF;
END$$
DELIMITER ;
```

O MySQL não tem ASSERTION: use CHECK e TRIGGER.

TRIGGER — TESTANDO

```
-- OK: devolução depois da retirada
INSERT INTO emprestimo VALUES
  (NULL, 'Dom Casmurro',
   '2026-09-01', '2026-09-15');
-- ERRO: devolução antes da retirada
INSERT INTO emprestimo VALUES
  (NULL, 'Quincas Borba',
   '2026-09-10', '2026-09-01');
```

Mensagem esperada: Data inválida (erro 1644).

RESUMO DO BLOCO 1

RESTRIÇÃO	COMANDO	ERRO PROVOCADO
DOMÍNIO	CHECK, NOT NULL	idade 200
ENTIDADE	PK, UNIQUE	CPF repetido
REFERENCIAL	FK, CASCADE	turma inexistente
USUÁRIO	CHECK, TRIGGER	devolução antes

Em todos os casos, o SGBD rejeita antes de gravar.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

DESAFIO DO BLOCO 1

UNIDADE 1 — INTEGRIDADE DE DADOS

1

Exercício orientado

Reescreva a tabela pedido com 4 restrições: chave primária, chave estrangeira, CHECK no valor e DEFAULT no status.

Resolução individual, com correção coletiva ao final.

PEDIDO — PONTO DE PARTIDA

```
CREATE TABLE cliente (
  id_cliente INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(80) NOT NULL
);
CREATE TABLE pedido_sem_regras (
  id INT, id_cliente INT, data_pedido DATE,
  valor DECIMAL(10,2), status VARCHAR(15)
);
```

Reescreva pedido_sem_regras com 4 restrições.

CORREÇÃO — PEDIDO COM REGRAS

```
CREATE TABLE pedido (
  id_pedido INT PRIMARY KEY AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  data_pedido DATE NOT NULL,
  valor DECIMAL(10,2) CHECK (valor > 0),
  status VARCHAR(15) DEFAULT 'ABERTO',
  FOREIGN KEY (id_cliente)
    REFERENCES cliente(id_cliente)
);
```

PK, FK, CHECK e DEFAULT em uma só tabela.

INTERVALO

15 minutos

Salve os scripts do Bloco 1

BLOCO 2 — SEGURANÇA

~75 minutos, em quatro etapas:

1. **SQL INJECTION**
   ~10 min — hash e prepared statement
2. **AUTENTICAÇÃO**
   ~12 min — CREATE USER e política de senha
3. **AUTORIZAÇÃO E DAC**
   ~23 min — GRANT, REVOKE, GRANT OPTION
4. **MAC E RBAC**
   ~30 min — view por nível, ROLE e desafio

SQL INJECTION — PREPARANDO

```
CREATE TABLE usuario (
  id INT PRIMARY KEY AUTO_INCREMENT,
  login VARCHAR(30) UNIQUE NOT NULL,
  senha_hash CHAR(64) NOT NULL
);
INSERT INTO usuario (login, senha_hash)
VALUES ('ana', SHA2('ana#123', 256)),
  ('admin', SHA2('Adm!n#2026', 256));
```

Guardamos o hash da senha, nunca a senha em texto.

SQL INJECTION — O ATAQUE

```
-- login digitado: ana
SELECT * FROM usuario
  WHERE login = 'ana';
-- login digitado: ' OR '1'='1
SELECT * FROM usuario
  WHERE login = '' OR '1'='1';
```

A consulta montada pelo sistema devolve TODOS os usuários.

SQL INJECTION — A DEFESA

```
PREPARE stmt FROM
  'SELECT id, login FROM usuario
   WHERE login = ?';
SET @login = "' OR '1'='1";
EXECUTE stmt USING @login;
SET @login = 'ana';
EXECUTE stmt USING @login;
DEALLOCATE PREPARE stmt;
```

Com parâmetro, o ataque vira só um login inexistente.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

AUTENTICAÇÃO — CRIANDO USUÁRIO

```
CREATE USER 'joao'@'localhost'
  IDENTIFIED BY 'SenhaForte#2025';
ALTER USER 'joao'@'localhost'
  IDENTIFIED BY 'NovaSenhaForte#2025';
SELECT user, host FROM mysql.user
  WHERE user = 'joao';
```

Autenticação: o joao já entra, mas ainda não pode nada.

AUTENTICAÇÃO — POLÍTICA DE SENHA

```
ALTER USER 'joao'@'localhost'
  PASSWORD EXPIRE INTERVAL 90 DAY;
ALTER USER 'joao'@'localhost'
  FAILED_LOGIN_ATTEMPTS 3
  PASSWORD_LOCK_TIME 1;
```

Senha vence em 90 dias; 3 erros bloqueiam a conta por 1 dia.

AUTENTICAÇÃO x AUTORIZAÇÃO

```
-- no terminal:
mysql -u joao -p
-- já dentro do MySQL, como joao:
SELECT CURRENT_USER();
SHOW DATABASES;
-- ERRO: SELECT command denied
SELECT * FROM escola.aluno;
```

Autenticado sim (entrou); autorizado não (sem privilégios).

AUTORIZAÇÃO — CONCEDENDO

```
GRANT SELECT, INSERT ON escola.aluno
  TO 'joao'@'localhost';
SHOW GRANTS FOR 'joao'@'localhost';
```

Privilégios no nível da tabela escola.aluno.

AUTORIZAÇÃO — TESTANDO O ACESSO

```
-- conectado como joao:
SELECT id_aluno, nome FROM escola.aluno;
INSERT INTO escola.aluno
  (cpf, nome, idade, email) VALUES
  ('44444444444', 'Duda', 16, 'd@e.com');
-- ERRO: UPDATE command denied
UPDATE escola.aluno SET nome = 'X'
  WHERE id_aluno = 1;
```

SELECT e INSERT funcionam; UPDATE não foi concedido.

AUTORIZAÇÃO — REVOGANDO

```
-- de volta ao administrador (root):
REVOKE INSERT ON escola.aluno
  FROM 'joao'@'localhost';
SHOW GRANTS FOR 'joao'@'localhost';
```

Agora o joao só consulta a tabela aluno.

GRANULARIDADE — POR COLUNA

```
CREATE USER 'maria'@'localhost'
  IDENTIFIED BY 'Maria#2025';
GRANT SELECT (id_aluno, nome, email)
  ON escola.aluno
  TO 'maria'@'localhost';
```

Privilégio só nas colunas id_aluno, nome e email.

GRANULARIDADE — TESTANDO

```
-- conectada como maria:
SELECT nome, email FROM escola.aluno;
-- ERRO: sem permissão na coluna cpf
SELECT cpf FROM escola.aluno;
```

O CPF fica protegido mesmo com acesso à tabela.

DAC — DONO CONCEDE ACESSO

```
CREATE USER 'prof_ana'@'localhost'
  IDENTIFIED BY 'ProfAna#2025';
CREATE USER 'prof_bia'@'localhost'
  IDENTIFIED BY 'ProfBia#2025';
GRANT SELECT, INSERT ON escola.aluno
  TO 'prof_ana'@'localhost'
  WITH GRANT OPTION;
```

WITH GRANT OPTION permite à ana repassar o acesso.

DAC — REPASSE PELA PROFESSORA

```
-- conectada como prof_ana:
GRANT SELECT ON escola.aluno
  TO 'prof_bia'@'localhost';
-- ERRO: a ana não possui DELETE
GRANT DELETE ON escola.aluno
  TO 'prof_bia'@'localhost';
```

Só se repassa o que se possui.

DAC — RETIRANDO O REPASSE

```
-- como administrador (root):
REVOKE GRANT OPTION ON escola.aluno
  FROM 'prof_ana'@'localhost';
SHOW GRANTS FOR 'prof_ana'@'localhost';
SHOW GRANTS FOR 'prof_bia'@'localhost';
```

A ana perde o repasse; a bia mantém o SELECT recebido.

MAC — DOCUMENTOS CLASSIFICADOS

```
CREATE TABLE documento (
  id INT PRIMARY KEY AUTO_INCREMENT,
  titulo VARCHAR(80) NOT NULL,
  nivel TINYINT NOT NULL
    CHECK (nivel BETWEEN 1 AND 3)
);
INSERT INTO documento (titulo, nivel)
  VALUES ('Cardápio', 1), ('Notas', 2),
         ('Ata secreta', 3);
```

Níveis: 1 público, 2 confidencial, 3 sigiloso.

MAC — HABILITAÇÃO DOS USUÁRIOS

```
CREATE TABLE habilitacao (
  usuario VARCHAR(30) PRIMARY KEY,
  nivel TINYINT NOT NULL
);
INSERT INTO habilitacao VALUES
  ('prof_ana', 2), ('prof_bia', 1);
```

A política central define o nível de cada usuário.

MAC — VIEW QUE APLICA A POLÍTICA

```
CREATE VIEW documento_permitido AS
SELECT d.id, d.titulo, d.nivel
FROM documento d
JOIN habilitacao h
  ON h.usuario =
     SUBSTRING_INDEX(USER(), '@', 1)
WHERE d.nivel <= h.nivel;
```

Cada usuário só enxerga níveis até a sua habilitação.

MAC — TESTANDO A POLÍTICA

```
GRANT SELECT ON escola.documento_permitido
  TO 'prof_ana'@'localhost';
GRANT SELECT ON escola.documento_permitido
  TO 'prof_bia'@'localhost';
-- ana vê níveis 1 e 2; bia vê só o 1
SELECT * FROM escola.documento_permitido;
```

O MySQL não tem MAC nativo: aqui foi simulado com view.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

RBAC — CRIANDO PAPÉIS

```
CREATE ROLE 'r_secretaria',
            'r_professor',
            'r_aluno';
```

Papéis (roles) agrupam privilégios e ainda não têm dono.

RBAC — PRIVILÉGIOS DOS PAPÉIS

```
GRANT SELECT, INSERT, UPDATE, DELETE
  ON escola.aluno TO 'r_secretaria';
GRANT SELECT ON escola.aluno
  TO 'r_professor';
GRANT SELECT ON escola.turma
  TO 'r_aluno';
```

Os privilégios vão para o papel, não para a pessoa.

RBAC — USUÁRIO RECEBE O PAPEL

```
CREATE USER 'sec_carla'@'localhost'
  IDENTIFIED BY 'Carla#2025';
GRANT 'r_secretaria'
  TO 'sec_carla'@'localhost';
SET DEFAULT ROLE 'r_secretaria'
  TO 'sec_carla'@'localhost';
```

A Carla herda tudo o que a secretaria pode fazer.

RBAC — VERIFICANDO E REVOGANDO

```
SHOW GRANTS FOR 'sec_carla'@'localhost'
  USING 'r_secretaria';
-- a Carla deixa a secretaria:
REVOKE 'r_secretaria'
  FROM 'sec_carla'@'localhost';
```

Trocar de função é mudar o papel, não refazer GRANTs.

QUANDO USAR CADA MODELO?

MODELO	QUEM DECIDE	PRATICAMOS
DAC	O dono do dado	WITH GRANT OPTION
MAC	Política central	view por nível
RBAC	O papel (role)	CREATE ROLE + GRANT

Empresas e escolas costumam adotar RBAC.

PREPARANDO O DESAFIO

```
CREATE TABLE nota (
  id_aluno INT, id_turma INT,
  valor DECIMAL(4,1)
    CHECK (valor BETWEEN 0 AND 10),
  PRIMARY KEY (id_aluno, id_turma)
);
```

Tabelas do desafio: aluno, turma, matricula e nota.

DESAFIO DO BLOCO 2

UNIDADE 1 — SEGURANÇA EM BANCO DE DADOS

2

Exercício orientado

Defina os privilégios de Secretaria, Professor e Aluno e escreva os GRANT dos três papéis, usando aluno, turma, matricula e nota.

Resolução individual, com correção coletiva ao final.

CORREÇÃO — SECRETARIA

```
-- Secretaria: cadastro completo
GRANT SELECT, INSERT, UPDATE, DELETE
  ON escola.aluno TO 'r_secretaria';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON escola.matricula TO 'r_secretaria';
GRANT SELECT, INSERT, UPDATE
  ON escola.turma TO 'r_secretaria';
```

A secretaria cadastra alunos, turmas e matrículas.

CORREÇÃO — PROFESSOR E ALUNO

```
-- Professor: consulta e lança notas
GRANT SELECT ON escola.aluno
  TO 'r_professor';
GRANT SELECT, INSERT, UPDATE
  ON escola.nota TO 'r_professor';
-- Aluno: só consulta
GRANT SELECT ON escola.nota
  TO 'r_aluno';
```

Na prática, o aluno só deve ver as próprias notas.

BLOCO 3 — PRIVACIDADE

~40 minutos, LGPD na prática:

1. **INVENTÁRIO E CRIPTOGRAFIA**
   ~10 min — dados sensíveis e AES
2. **PSEUDONIMIZAÇÃO E ANONIMIZAÇÃO**
   ~8 min — mapa e relatório
3. **AUDITORIA E DIREITOS**
   ~12 min — trigger e comandos da LGPD
4. **DESAFIO FINAL**
   ~10 min — mascaramento de dados

LGPD — TABELA DE TITULARES

```
CREATE TABLE titular (
  id_titular INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(80) NOT NULL,
  cpf CHAR(11) UNIQUE NOT NULL,
  email VARCHAR(100) NOT NULL,
  cidade VARCHAR(40),
  saude VARBINARY(255),
  consentimento TINYINT(1) DEFAULT 0
);
```

saude é dado sensível: será guardado criptografado.

INVENTÁRIO DE DADOS

```
CREATE TABLE inventario_dados (
  coluna VARCHAR(30) PRIMARY KEY,
  tipo VARCHAR(10) NOT NULL
    CHECK (tipo IN ('PESSOAL','SENSIVEL')),
  finalidade VARCHAR(60) NOT NULL
);
```

Princípio da finalidade: cada dado tem um motivo registrado.

INVENTÁRIO — PREENCHENDO

```
INSERT INTO inventario_dados VALUES
  ('nome', 'PESSOAL', 'identificar'),
  ('cpf', 'PESSOAL', 'identificar'),
  ('email', 'PESSOAL', 'contato'),
  ('saude', 'SENSIVEL', 'atendimento');
SELECT * FROM inventario_dados;
```

Dados pessoais x sensíveis, com finalidade declarada.

CRIPTOGRAFIA — GRAVANDO

```
SET @k = SHA2('chave-mestra', 256);
INSERT INTO titular (nome, cpf, email,
  cidade, saude, consentimento) VALUES
 ('Ana','11111111111','ana@mail.com',
  'Rio Verde',AES_ENCRYPT('asma',@k),1),
 ('Bia','22222222222','bia@mail.com',
  'Jataí',AES_ENCRYPT('rinite',@k),0);
```

A chave (@k) vive fora do banco; aqui é só demonstração.

CRIPTOGRAFIA — LENDO OS DADOS

```
-- sem a chave: conteúdo ilegível
SELECT nome, saude FROM titular;
-- com a chave: conteúdo legível
SELECT nome,
  CAST(AES_DECRYPT(saude, @k) AS CHAR)
    AS saude
FROM titular;
```

Só quem tem a chave lê o dado sensível.

PSEUDONIMIZAÇÃO — O MAPA

```
CREATE TABLE mapa_pseudonimo (
  codigo CHAR(10) PRIMARY KEY,
  id_titular INT UNIQUE NOT NULL,
  FOREIGN KEY (id_titular)
    REFERENCES titular(id_titular)
    ON DELETE CASCADE
);
```

O mapa fica separado, com acesso restrito ao controlador.

PSEUDONIMIZAÇÃO — GERANDO CÓDIGOS

```
SET @sal = '#segredo';
INSERT INTO mapa_pseudonimo
SELECT LEFT(SHA2(CONCAT(cpf,@sal),256),10),
       id_titular
FROM titular;
SELECT * FROM mapa_pseudonimo;
```

Código derivado do CPF + segredo: só reverte quem tem o mapa.

PROF. JÚLIO
ANALISTA DE
SISTEMAS

View em SQL

Uma **view** é uma tabela virtual definida por uma consulta SELECT armazenada no banco. Ela não guarda dados próprios (exceto as materializadas): ao ser consultada, executa a query original.

View em SQL

Uso e características

Para que é usada

Simplificar consultas complexas (joins, agregações)

Restringir acesso a colunas/linhas (segurança)

Padronizar regras de negócio e relatórios

Manter compatibilidade quando a estrutura das tabelas muda

View em SQL

Características

Criada com CREATE VIEW nome AS SELECT ...

Consultada como uma tabela comum

Reflete sempre os dados atuais das tabelas base

Pode ser atualizável (se simples) ou somente leitura

Existe a versão *materializada*, que armazena o resultado

View em SQL

Pontos fortes

Reuso de lógica: escreve uma vez, usa em vários lugares

Segurança: expõe só o necessário (ex.: oculta salário)

Abstração: o usuário não precisa conhecer o esquema

Facilita manutenção e organização do código SQL

View em SQL

Pontos fracos

Pode prejudicar o desempenho (views aninhadas, queries pesadas)

Atualização (INSERT/UPDATE) limitada em views complexas

Dependência das tabelas base: alterações podem quebrá-las

Views materializadas exigem refresh e ocupam espaço

PSEUDONIMIZAÇÃO — VISÃO ANALÍTICA

```
CREATE VIEW analise_pseudonimizada AS
SELECT m.codigo, t.cidade
FROM titular t
JOIN mapa_pseudonimo m USING (id_titular);
SELECT * FROM analise_pseudonimizada;
```

O analista vê códigos e cidade, sem nome nem CPF.

ANONIMIZAÇÃO — SEM VOLTA

```
CREATE TABLE relatorio_anonimo AS
SELECT cidade, COUNT(*) AS total
FROM titular
GROUP BY cidade;
SELECT * FROM relatorio_anonimo;
```

Sem nome, CPF ou código: não há como voltar ao titular.

AUDITORIA — TABELA DE LOG

```
CREATE TABLE log_auditoria (
  id INT PRIMARY KEY AUTO_INCREMENT,
  usuario VARCHAR(100) NOT NULL,
  acao VARCHAR(10) NOT NULL,
  id_titular INT,
  data_hora DATETIME NOT NULL
    DEFAULT CURRENT_TIMESTAMP
);
```

Registra quem fez o quê e quando.

AUDITORIA — TRIGGER (1/2)

```
DELIMITER $$
CREATE TRIGGER audit_titular_upd
AFTER UPDATE ON titular
FOR EACH ROW
BEGIN
```

O trigger dispara depois de cada UPDATE em titular.

AUDITORIA — TRIGGER (2/2)

```
  INSERT INTO log_auditoria
    (usuario, acao, id_titular)
  VALUES
    (CURRENT_USER(), 'UPDATE',
     NEW.id_titular);
END$$
DELIMITER ;
```

NEW.id_titular é o id da linha alterada.

AUDITORIA — TESTANDO

```
UPDATE titular
   SET email = 'ana.souza@mail.com'
   WHERE id_titular = 1;
SELECT usuario, acao, id_titular, data_hora
FROM log_auditoria;
```

Triggers não registram SELECT: use um plugin de auditoria.

DIREITOS — CONFIRMAÇÃO E ACESSO

```
-- 1) Confirmação do tratamento
SELECT COUNT(*) AS existe
FROM titular WHERE cpf = '11111111111';
-- 2) Acesso aos dados
SELECT nome, cpf, email, cidade
FROM titular WHERE cpf = '11111111111';
```

Direitos do titular: confirmar e acessar os dados.

DIREITOS — CORREÇÃO E PORTABILIDADE

```
-- 3) Correção
UPDATE titular SET cidade = 'Jataí'
  WHERE cpf = '11111111111';
-- 4) Portabilidade (formato JSON)
SELECT JSON_OBJECT('nome', nome,
  'email', email, 'cidade', cidade)
FROM titular WHERE cpf = '11111111111';
```

Portabilidade: entregar os dados em formato estruturado.

DIREITOS — ELIMINAÇÃO

```
-- 5) Eliminação dos dados
DELETE FROM titular
  WHERE cpf = '22222222222';
SELECT * FROM mapa_pseudonimo;
SELECT * FROM titular;
```

O DELETE CASCADE também apagou o pseudônimo da Bia.

DESAFIO FINAL

UNIDADE 1 — PRIVACIDADE DE DADOS E LGPD

3

Exercício orientado

Escolha um sistema (escola ou e-commerce): 1) dados pessoais e sensíveis; 2) titular, controlador e operador; 3) uma medida técnica em SQL.

Resolução individual, com correção coletiva ao final.

EXEMPLO — MASCARAMENTO DE DADOS

```
CREATE VIEW titular_mascarado AS
SELECT nome,
  CONCAT(LEFT(email, 2), '***@',
    SUBSTRING_INDEX(email, '@', -1)) AS email,
  CONCAT('*******', RIGHT(cpf, 4)) AS cpf
FROM titular;
SELECT * FROM titular_mascarado;
```

Mascaramento aplica o princípio da necessidade.

SÍNTESE DA AULA

1. **INTEGRIDADE**
   CHECK, PK, FK, UNIQUE e TRIGGER

2. **SEGURANÇA**
   CREATE USER, GRANT, REVOKE e ROLE

3. **PRIVACIDADE E LGPD**
   AES, hash, auditoria e direitos

Refaça os scripts do Word e os desafios em casa.

PRÓXIMO ENCONTRO

- Unidade 2: Elaboração e Implementação de um Projeto de Sistemas de Banco de Dados
- Fases de um projeto de banco de dados e levantamento de requisitos

Lembrete: 20h da carga horária são atividades direcionadas — exercícios de SQL resolvidos fora da aula.